Project Idea / Concept

Project Name: Lamsat - Homemade E-Commerce Platform

Problem Statement: In Egypt, thousands of talented homemade artists and craftspeople struggle to sell their work online. Existing platforms like Facebook and Instagram offer no buyer or seller protection, leaving both parties vulnerable to fraud and scams. Additionally, the absence of integrated and trusted online payment methods forces transactions to rely on informal cash exchanges, creating friction and distrust. As a result, artists cannot scale their work into real businesses, and buyers hesitate to purchase from unknown sellers.

Proposed Solution: Lamsat addresses this gap by providing a dedicated, trusted marketplace for Egyptian homemade artists with built-in fraud protection and seamless online payment integration.

Target Audience: Homemade artists and craftspeople looking to sell their creations online, and Egyptian consumers aged 18–40 who seek unique, personalized, and handmade products as gifts or for personal use.

Unique Value Proposition(UVP): Unlike general e-commerce platforms like Jumia or informal social media selling on Facebook and Instagram, Lamsat is Egypt's first dedicated handmade marketplace that combines fraud protection, integrated online payments, and a verified artist storefront giving both buyers and sellers a safe, trusted, and seamless experience tailored for the Egyptian market.

Core Features:

- Role-Based Authentication (Admin, Buyer, Seller)
- Seller Storefront Creation and Management
- Product Listing with Images, Description, and Pricing
- Shopping Cart with Session Management
- Secure Checkout via Paymob
- Fraud Protection and Escrow Payment System
- Order Tracking for Buyers and Sellers
- Reviews and Ratings System
- Admin Dashboard for User, Order Management
- Seller Verification and Approval System

Team Members **Team Size = 6 members - communication plan: WhatsApp Group**

Yousef Waleed - The Project Lead & Architect

Rule: You own the project’s high-level structure. You initialize the N-Tier layers, approve all Git Pull Requests, and ensure every teammate follows the Repository and Unit of Work patterns to keep the code clean and scalable.

Ahmed Ahmed - The Database & API Logic (A)

Rule: You own the core Product Engine. You design the SQL Server tables for categories and products, write the LINQ queries to fetch them, and build the Web API controllers that allow the frontend to search and filter handmade goods.

Rahma Sherif - The Database & API Logic **(B)**

Rule: You own the Order & Cart lifecycle. You manage the database logic for shopping carts and order history, ensuring that stock levels decrease correctly when a purchase is made and that order statuses are tracked from "Pending" to "Delivered."

Ahmed Mohamed - The Identity & Payments Specialist

Rule: You own the Gatekeeping. You set up ASP.NET Identity (JWT) for secure login and handle the Paymob integration, ensuring the 10% platform commission is calculated correctly and funds are held securely until delivery is confirmed.

Fathi Mohsen - The Frontend & UI Designers **(A)**

Rule: You own the Storefront UI. Using Bootstrap and CSS, you build the responsive home page, product catalogs, and the checkout journey, ensuring the site looks professional and high-contrast on all devices.

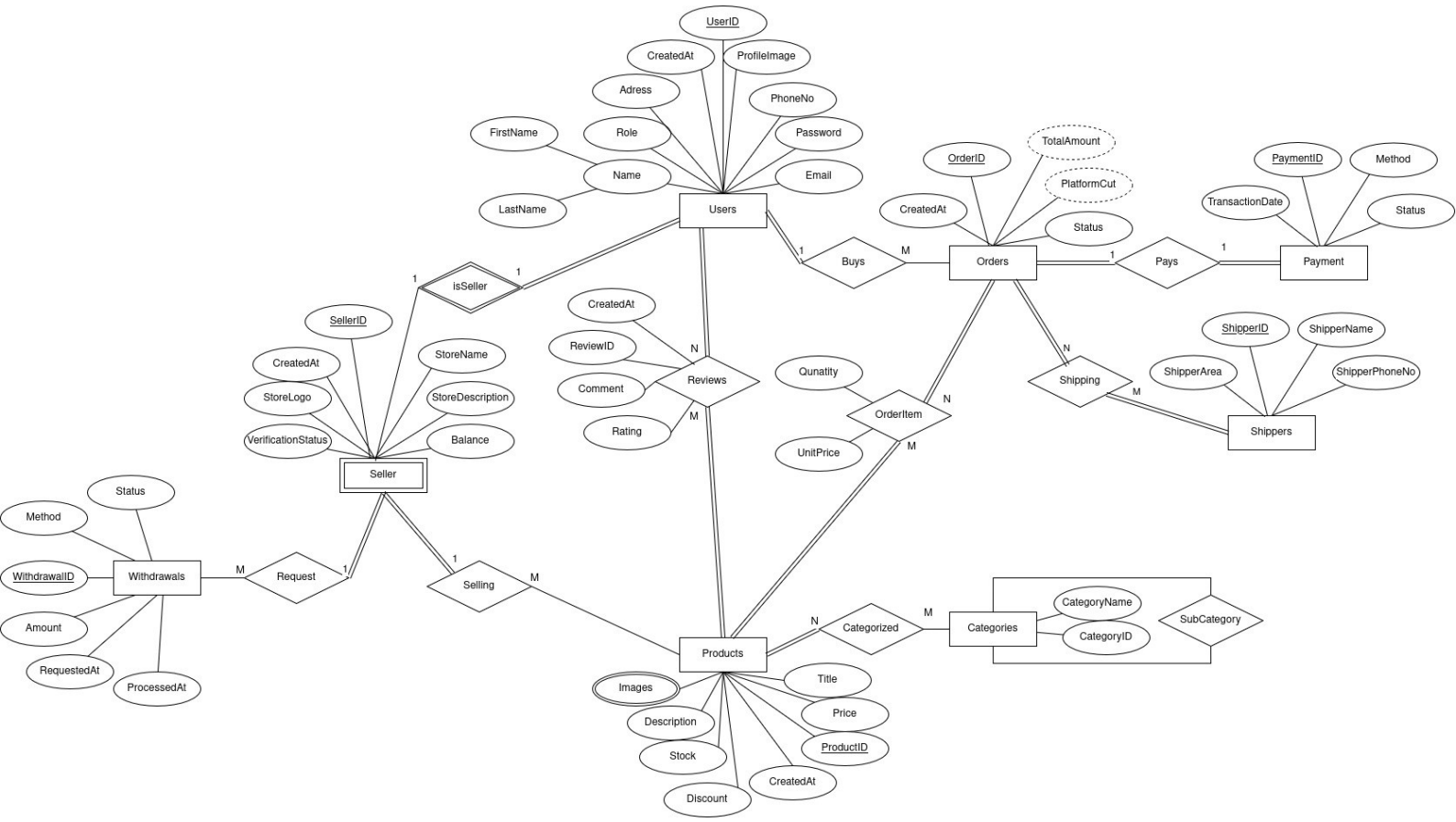
Kyrillos Anis - The Frontend & UI Designers **(B)**

Rule: You own the Management Dashboards. You build the specialized views for Admins to approve sellers and for Sellers to manage their "Wallet" and product listings, using DataTables to make large amounts of information easy to navigate.

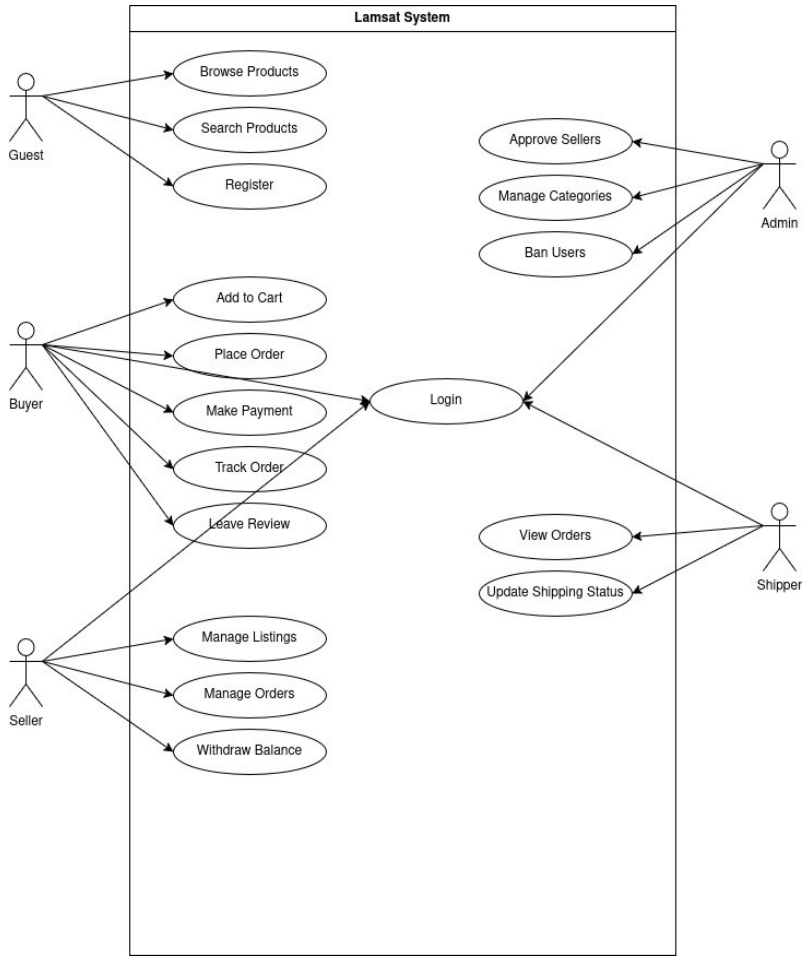
System Analysis

- **Functional Requirements**
  - **Guest Users can:**
    - Browse and search products by category or keyword
    - View product details and seller profiles
    - Register as a Buyer or Seller
  - **Buyers can:**
    - Add products to cart and wishlist
    - Place and track orders
    - Pay securely via Paymob
    - Confirm order delivery
    - Leave reviews and ratings
    - Open a dispute if there's an issue
  - **Sellers can:**
    - Apply and get verified by Admin
    - Create and manage their storefront
    - Add, edit, and delete product listings
    - View and manage incoming orders
    - Track their balance (90% per sale)
    - Request withdrawal
  - **Admins can:**
    - Approve or reject seller applications
    - Manage categories and platform content
    - Handle disputes between buyers and sellers
    - Ban users or remove violating listings
    - View platform analytics and reports
- **Non-Functional Requirements**
  - **Performance** The system shall load pages in under 3 seconds and support at least 500 concurrent users without degradation.
  - **Security** Passwords must be hashed using BCrypt. All communication must be over HTTPS. Authentication is handled via JWT tokens. Payment data is managed entirely by Paymob and never stored on Lamsat's servers.
  - **Scalability** The system architecture must support growth in users, products, and orders without requiring major redesign.
  - **Availability** The platform must maintain 99.5% uptime with automated daily database backups.
  - **Usability** The UI must be mobile-responsive and support both Arabic and English content.
  - **Reliability** All financial transactions must be logged. Order status changes must be tracked with timestamps.
  - **Maintainability** Code must follow clean architecture principles with clear separation between layers (API, Core, Infrastructure).

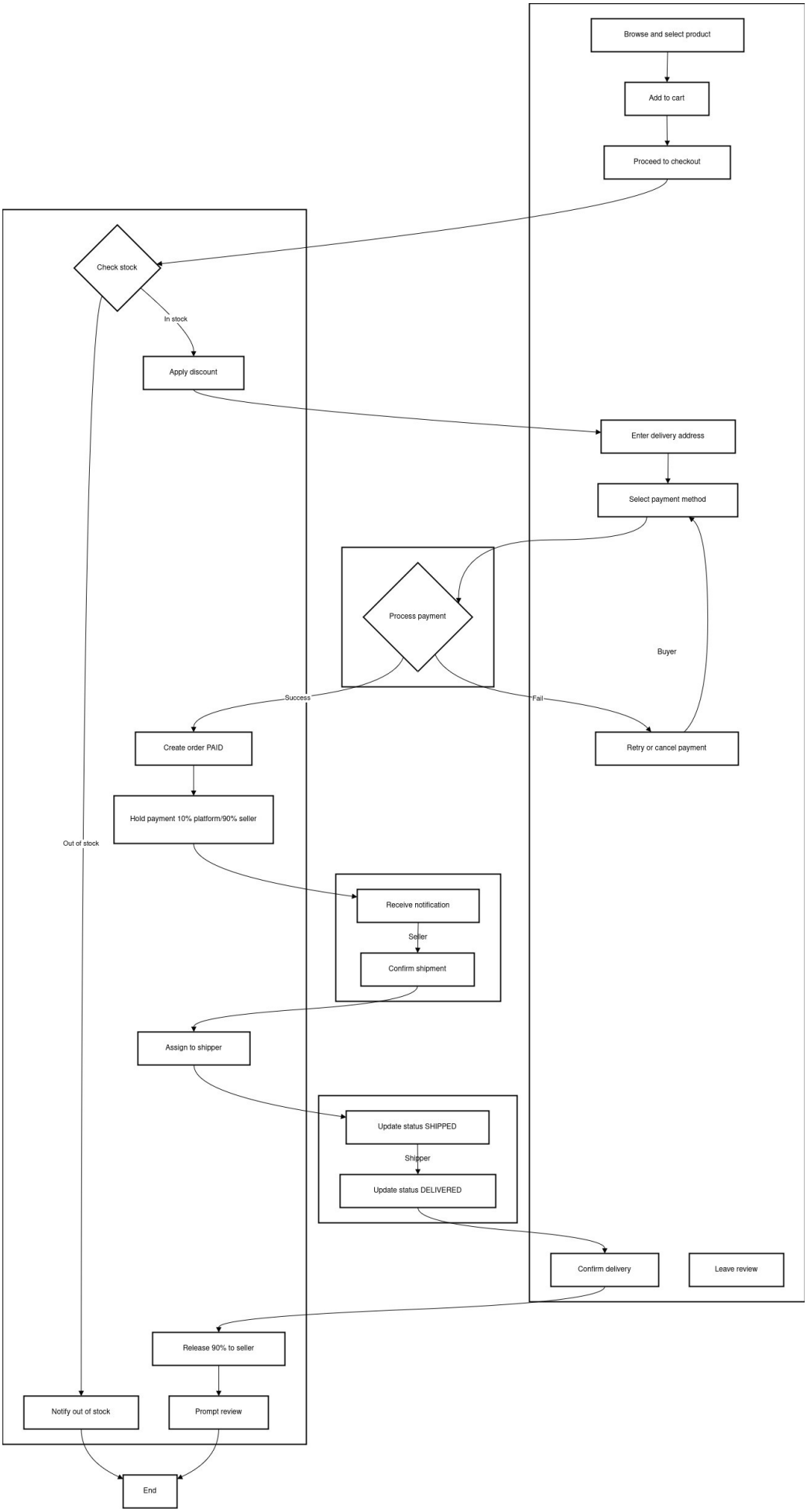
- **Database Design (ERD Entities):**



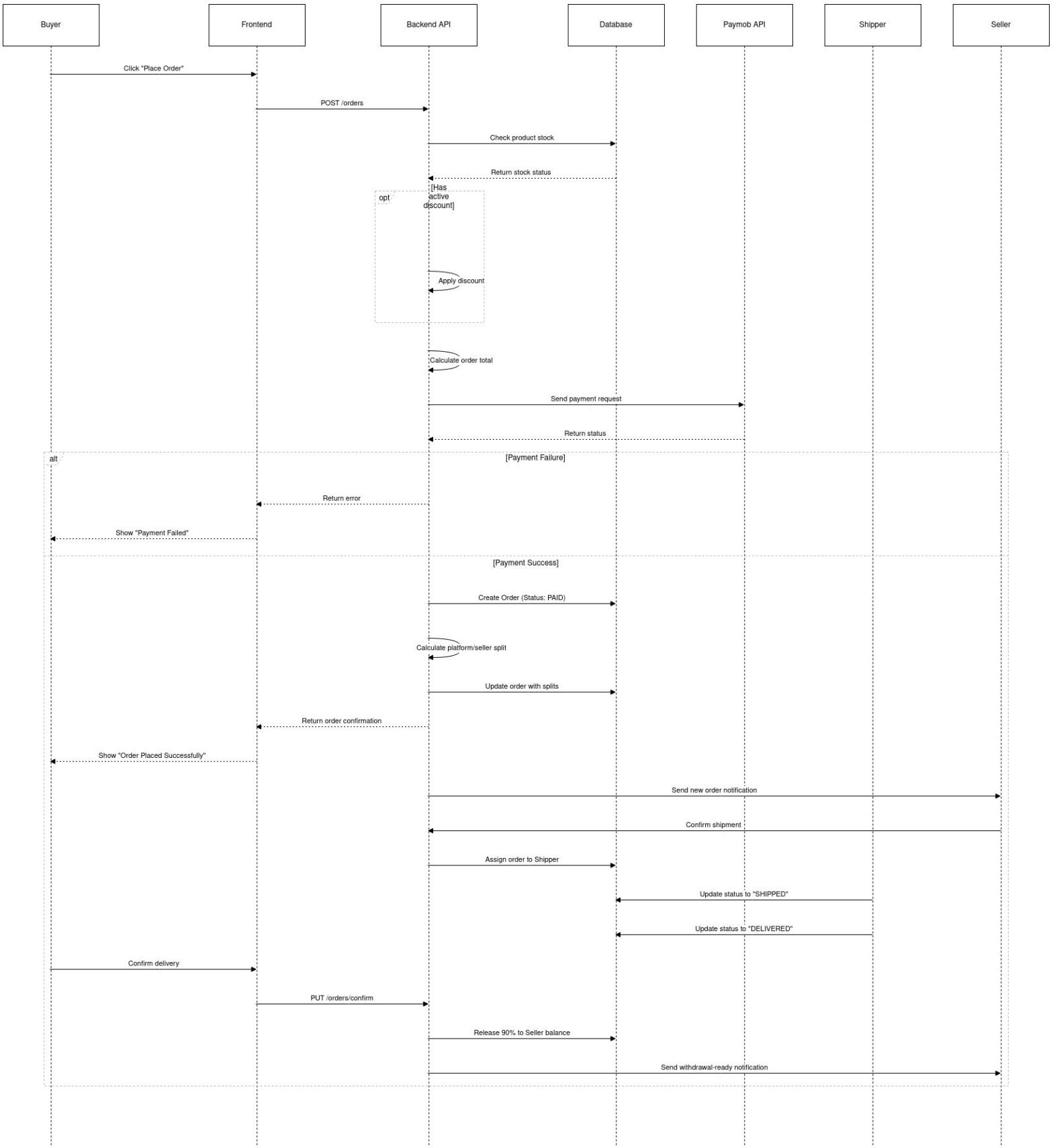
- **Use case Diagram:**



- **Activity Diagram:**



- **Sequence Diagram:**



- **Case Study / Feasibility Study**
  - **Technical Feasibility:**
    - **Question:** Do we have the technology to build this?
      - **Answer:** Yes. Using ASP.NET Core Web API allows rapid backend development. Entity Framework Core speeds up database creation and migrations. Paymob provides a ready-to-use sandbox API for payments so we don't have to build a payment engine from scratch. Bootstrap and plain HTML/CSS/JS handle the frontend with no additional learning curve for the team.
  - **Operational Feasibility:**
    - **Question:** Will end-users actually adopt and use this system?
      - **Answer:** Yes, for the following reasons:
    - **For Buyers:** The interface is built with plain HTML/CSS/JS and Bootstrap, making it fast, lightweight, and accessible on any device including low-end mobile phones which are the primary browsing device in Egypt. The familiar e-commerce layout (search, cart, checkout) requires no learning curve.
    - **For Sellers:** The seller dashboard is straightforward adding a product is as simple as filling a form with a photo, title, price, and description. No technical knowledge is required to operate a storefront on Lamsat.
    - **For Admins:** The admin panel provides a centralized place to manage users, sellers, disputes, and analytics reducing manual effort and making platform management efficient for a small team.
    - **Trust Factor:** The verified seller badges, review system, and escrow-based payment flow directly address the biggest reason Egyptians avoid online shopping from unknown sellers lack of trust. This makes adoption significantly more likely compared to informal alternatives like Facebook Marketplace.
  - **Economic Feasibility:**
    - **Costs:** Domain (~$10/year), Cloudinary free tier for image storage, Paymob sandbox (free), Railway or Azure free tier for hosting. Total cost is nearly $0 for the MVP.
    - **Benefits:** The 10% commission model means Lamsat earns on every transaction without holding any inventory. Automating payments and dispute management reduces the need for manual staff. The ROI is highly positive even at a small scale.
  - **Competitor Case Study (Egypt’s Market vs. Lamsat):**
    - **What Facebook Marketplace does well:** Large existing user base, free to use, easy to post.
    - **Where Facebook Marketplace fails:** No buyer protection, no payment integration, no verified sellers, and rampant scamming makes buyers hesitant to purchase from unknown sellers.
    - **Our Edge:** Lamsat is exclusively dedicated to handmade and artisan products with verified seller profiles, a secure escrow-based payment flow via Paymob, and a built-in dispute resolution system giving both buyers and sellers a level of trust that no current Egyptian platform offers.

- **Risk Assessment:**
  - **Question:** What are the potential risks that could affect the project and how will we handle them?
    - **Answer:** We have identified the following risks and backup plans:
  - **Risk 1 — Running out of time:**
    - Risk: The project has a fixed graduation deadline and 6 features might not all be completed.
    - Backup Plan: We prioritized an MVP scope. Core features (auth, listings, cart, payment, orders) are built first. Secondary features (withdrawals, analytics, disputes) are built after. If time runs short, we deliver a working core.
  - **Risk 2 — Paymob API failure or integration issues:**
    - Risk: Paymob sandbox may have downtime or unexpected behavior during testing or demo.
    - Backup Plan: We simulate the payment flow with a mock payment option (cash on delivery) as a fallback so the rest of the system still functions during demo.
  - **Risk 3 — Team member dropping out or becoming unavailable:**
    - Risk: One or more team members may become unavailable due to personal reasons.
    - Backup Plan: All code is on GitHub with clear documentation. Any team member can pick up another's tasks. No single person owns a critical feature alone.
  - **Risk 4 — Scope creep:**
    - Risk: Adding too many features beyond the original plan delays delivery.
    - Backup Plan: A fixed feature list is agreed upon at the start. Any new feature suggestions are logged for a future version, not added mid-development.
  - **Risk 5 — Database design flaws discovered late:**
    - Risk: A missing relationship or wrong attribute discovered after development starts causes major refactoring.
    - Backup Plan: Full ERD and schema are reviewed and approved by the whole team before any code is written. Entity Framework migrations make schema changes manageable if needed.
  - **Risk 6 — Hosting or deployment issues:**
    - Risk: The app fails to deploy properly for the demo day.
    - Backup Plan: We run the project locally as a fallback. A localhost demo is prepared as a backup in case cloud deployment fails.

- UI/UX design
  - **Figma**